- INFINITY—The representation of infinity.
- MAX—The largest possible value in a double-precision floating point.
- MAX_EXP—The largest exponent in a double-precision floating point; should be 1024.
- MIN—The smallest possible value in a double-precision floating point.
- MIN_EXP—The smallest exponent in a double-precision floating point; should be 1021.
- NAN—Represents not a number, usually the result of zero divided by zero.

Floats implement the basic arithmetic operators, +, -, *, /, %, and **, and have a query operator for nan?.

Floats can't quite be in ranges, but they do have next_float and prev_float methods that return the adjacent representable float values.

Floats can be approximately converted to a rational with the rationalize method, and have to_d, to_f, to_i and to_int (both of which truncate the float to an integer), to_r and to_s.

Beyond that, the Float class has the same features as a generic Numeric object.

Rational Class

The Rational class is a subclass of Numeric. Rationals can be created with the literal syntax, (see Rational and Complex Numbers, on page 434) or with the Kernel method Rational, which takes the numerator and denominator as arguments.

Nearly all the methods of Rational are performance improvements over methods that exist in Numeric. The arithmetic methods +, -, *, and /, and ** are defined. The conversion methods to_d, to_f, to_i, to_json, to_r, and to_s are defined.

The surprise there is to_json, which returns a JSON representation of the hash {"json_class" => "Rational", "n" => self.numerator, "d" => self.denominator}. (See *Chapter 28, Library Reference: I/O* for more information.) The JSON string can be deserialized with the class method Rational.json_create.

Complex Class

The Complex class is a subclass of Numeric. Complex numbers can be created with the literal syntax (see Rational and Complex Numbers, on page 434) or with the Kernel method Complex, which takes the real and imaginary parts as arguments. The Complex class also defines Complex.polar and Complex.rectangular methods that convert from polar to rectangular formats for identifying complex numbers.

The real part has a getter method real; the imaginary part has a getter imaginary, which is aliased as imag. The method rectangular or rect returns a two-element array: [real, imaginary]. For complex numbers, the query method real? always returns false, no matter the value of the imaginary part.

In polar form, the getters are magnitude (aliased as abs) and phase (aliased as arg). The getter polar returns the two polar values as a two-element array, [magnitude, phase]

The infinite? boolean will return true if either the real or imaginary part of the number is infinite.

Nearly all the methods of Complex are performance improvements over methods that exist in Numeric. The arithmetic methods +, -, *, and /, and ** are defined.